

Lista 06 – Modularização: Funções

Revisão

- **Definição:** Usamos a palavra reservada `def` para criar uma função, seguida do nome, parênteses (Argumentos) e dois pontos “:”.
- **Argumentos:** São os dados que a função recebe (colocados dentro dos parênteses).
- **Retorno (return):** É a resposta que a função devolve para o programa principal. Uma função pode imprimir algo ou *retornar* um valor para ser usado depois.
- **Valores Default (Padrão):** Podemos definir valores opcionais para os argumentos. Se o usuário não enviar o dado, a função usa o valor padrão. Ex: `def oi(nome="Visitante"):`

- Exercício 1

Crie uma função que receba duas strings como parâmetros e imprima uma mensagem formatada de boas-vindas. O programa principal deve pedir esses dados ao usuário e chamar a função.

Exemplo de execução:

Entrada:

Digite seu nome: Gabriel

Digite seu curso: Ciência da Computação

Saída Esperada:

Olá Gabriel! Seja bem-vindo(a) ao curso de Ciência da Computação.

- Exercício 2

Crie uma função que receba um número inteiro e retorne o valor booleano `True` se for maior que zero, e `False` caso contrário. Teste a função no programa principal.

Exemplo de execução:

Entrada:

Digite um número: -5

Saída Esperada:

O número não é positivo.

- Exercício 3

Crie uma função que receba um número inteiro positivo, calcule e retorne o seu fatorial usando um laço de repetição. Lembre-se que o fatorial de 0 é 1.

Exemplo de execução:

Entrada:

Digite um número para calcular o fatorial: 5

Saída Esperada:

O fatorial de 5 é 120

- Exercício 4

Crie uma função que receba uma lista contendo vários números inteiros. A função deve retornar uma nova lista contendo apenas os números pares da lista original.

Exemplo de execução:

Entrada:

[1, 2, 3, 4, 5, 6]

Saída Esperada:

Lista original: [1, 2, 3, 4, 5, 6]

Lista filtrada (apenas pares): [2, 4, 6]

- Exercício 5

Crie uma função que recebe uma lista de números inteiros e retorna, de uma só vez, os dois valores: o maior e o menor número da lista. No programa principal, capture esses dois retornos em duas variáveis separadas e exiba.

Exemplo de execução:

Entrada:

[45, 12, 89, 2, 34]

Saída Esperada:

O maior é 89 e o menor é 2

- Exercício 6

Construa uma calculadora utilizando o conceito de modularização. Crie quatro funções matemáticas: [1] soma(a, b), [2] subtrai(a, b), [3] multiplica(a, b) e [4] divide(a, b). No programa principal, crie um laço com um Menu interativo para o usuário escolher a operação. De acordo com a escolha, peça os números, chame a função correta e exiba o resultado. Não se esqueça de tratar a divisão por zero.

Exemplo de execução:

Entrada:

(Menu exibido)

Escolha uma opção: 4

Primeiro número: 10

Segundo número: 2

Saída Esperada:

Resultado: 5.0

- Exercício 7

Crie uma função valida_senha(senha) que recebe uma string. A função deve retornar True se a senha for forte, e False caso contrário. Regras para senha forte:

1. Ter no mínimo 8 caracteres.
2. Não conter espaços em branco.
3. Conter pelo menos um caractere especial: "@", "!", ou "#".

Utilize a nova função no programa principal.

Exemplo de execução:

Entrada:

Digite a senha para validação: UFPR!2026

Saída Esperada:

Aprovado: Senha Forte!

- Exercício 8

Crie duas funções: Uma que cria uma matriz com os valores que o usuário digitar e uma que recebe essa matriz (lista de listas) e a imprime na tela formatada linha por linha (não apenas usando o print bruto da lista).

Exemplo de execução:

Entrada:

Valor [0][0]: 1

Valor [0][1]: 2

Valor [0][2]: 3

Valor [1][0]: 4

Valor [1][1]: 5

Valor [1][2]: 6

Valor [2][0]: 7

Valor [2][1]: 8

Valor [2][2]: 9

Saída Esperada:

| 1 | 2 | 3 |

| 4 | 5 | 6 |

| 7 | 8 | 9 |

- Desafio 1

A lógica por trás dos números romanos exige olhar não apenas para a letra atual, mas para a próxima. Crie uma função `romano_para_inteiro(romano)` que receba uma string representando um número romano e retorne o seu valor inteiro. As letras têm valores fixos (I=1, V=5, X=10, L=50, C=100, D=500, M=1000). Se uma letra de menor valor vem *antes* de uma de maior valor, ela deve ser subtraída (Ex: "IV" = 4). Caso contrário, é somada (Ex: "VI" = 6). Utilize quantas funções auxiliares forem necessárias.

Exemplo de execução:

Entrada:

Digite um número romano: XIV

Saída Esperada:

O valor inteiro de XIV é 14

- Desafio 2

Em qualquer sistema comercial ou banco de dados no Brasil, validar um CPF é uma etapa obrigatória. Um CPF não é apenas validado pelo seu tamanho (11 dígitos), mas por um cálculo matemático rigoroso que gera os dois últimos "Dígitos Verificadores". Crie um programa utilizando modularização contendo duas funções:

1. `limpa_cpf(cpf_sujo)`: Recebe a string que o usuário digitou e devolve apenas os números, removendo pontos e traços.
2. `valida_cpf(cpf)`: Implementa o algoritmo oficial de validação.

Algoritmo do Dígito Verificador:

- Primeiro Dígito: Multiplique os 9 primeiros números pelos pesos decrescentes de 10 até 2 e some tudo. Multiplique a soma por 10 e divida por 11. O resto dessa divisão é o dígito (se o resto for 10, o dígito vira 0). Verifique se ele bate com o 10º número do CPF.
- Segundo Dígito: Mesma lógica, mas agora usa os 10 primeiros números multiplicados pelos pesos de 11 até 2. O resto da divisão dita o último dígito.

Exemplo de execução:

Entrada:

Digite um CPF (com ou sem pontuação): 111.111.111-11

Saída Esperada:

Status: CPF INVÁLIDO.